Tarea 4 EL4106 - Semestre Otoño 2024

Profesor: Javier Ruiz del Solar
Auxiliar: Patricio Loncomilla

Fecha enunciado: 22 de Mayo de 2024
Plazo entrega tarea: 3 de Junio de 2024

El objetivo de esta tarea es utilizar redes neuronales en un problema de clasificación de dígitos. Se utilizará el conjunto de datos *Optical Recognition of Handwritten Digits Data Set.* Este conjunto tiene 64 características, con 10 clases y 5620 muestras en total. La base de datos estará disponible en u-cursos.

Las redes a ser entrenadas tienen la siguiente estructura: capa de entrada de dimensionalidad 64 (correspondiente a los datos de entrada), capas ocultas (una o dos) y capa de salida con 10 neuronas y función de activación *softmax*. La función de *loss* (pérdida) es entropía cruzada. El optimizador que se debe usar es *Adam*. La función *softmax* está implícita al usar la función de pérdida *CrossEntropyLoss* de pytorch (no se debe agregar *softmax* a la salida de la red).

Se usará pytorch para entrenar y validar la red neuronal que implementa el clasificador de dígitos. Se analizará los efectos de cambiar el tamaño de la red (número de capas ocultas y de neuronas en estas capas) y la función de activación.

Se debe notar que hay múltiples hiperparámetros que hay que definir al entrenar una red. En particular, hay hiperparámetros asociados a la estructura de la red: número de capas, número de neuronas en cada capa y funciones de activación, así como hiperparámetros del entrenamiento, entre los cuales se cuentan el tamaño del minibatch y la tasa de aprendizaje. En esta tarea se variarán los hiperparámetros de la estructura de la red, mientras que se usará un tamaño de minibatch de 128 y una tasa de aprendizaje de 0.001.

En esta tarea se entrega un código base, el cual debe ser usado para realizar las actividades pedidas.

Se pide:

1) Ejecutar el código base entregado y verificar que este funciona adecuadamente. Este código carga el archivo “1_digits_train.txt” y “1_digits_test” en dos *DataFrames*, los cuales son usados a continuación para generar tres objetos *Dataset* (entrenamiento, validación y prueba). Los conjuntos de entrenamiento, validación y prueba tienen *DataLoader*s, los cuales permiten acceder a los elementos de los *Dataset*s mediante *batches*. Además, los datos se normalizan. Finalmente, el código base permite entrenar una red neuronal, aunque se deberán implementar las funcionalidades que eviten el sobreajuste (*overfitting*) de la misma.

2) Entrenar una red neuronal para cada uno de los siguientes casos:
   a) 10 neuronas en la capa oculta, función de activación ReLU y 1000 épocas
   b) 40 neuronas en la capa oculta, función de activación ReLU y 1000 épocas
   c) 10 neuronas en la capa oculta, función de activación Tanh y 1000 épocas
   d) 40 neuronas en la capa oculta, función de activación Tanh y 1000 épocas
   e) 2 capas ocultas con 10 neuronas cada una, función de activación ReLU y 1000 épocas
   f) 2 capas ocultas con 40 neuronas cada una, función de activación ReLU y 1000 épocas

   Se debe evitar el sobreajuste (*overfitting*) de la red. Para esto se debe detener el entrenamiento cuando el *loss* de validación comience a aumentar, mientras el de entrenamiento sigue bajando. Como las mediciones de *loss* pueden tener fluctuaciones (ruido), en lapsos cortos de tiempo, la implementación de esta funcionalidad debe realizarse en forma robusta, usando un enfoque basado en *patience*. (busque sobre *patience early stopping* en internet). El código guarda *checkpoints* para poder recuperar las redes correspondientes a distintas épocas de entrenamiento.

Para cada red a ser entrenada:

- Calcular el *loss* de entrenamiento y el de validación, así como el tiempo de entrenamiento total requerido. El entrenamiento se debe finalizar cuando el *loss* de validación comience a subir, para evitar sobreajuste de la red.
- Graficar el *loss* de entrenamiento y el de validación en función del número de épocas. Ambas curvas deben mostrarse en un mismo gráfico, colocando las leyendas adecuadas.
- Generar la matriz de confusión normalizada y el *accuracy*, usando el conjunto de entrenamiento. Se debe usar el *checkpoint* anterior al sobreajuste. Se debe mostrar la matriz de confusión con distintos colores o tonos de gris según el valor del *accuracy*. La matriz se debe calcular usando *scikit-learn*, usando la opción `normalize = "true"`.
- Generar la matriz de confusión normalizada y el *accuracy* normalizado, usando el conjunto de validación. Se debe usar el *checkpoint* anterior al sobreajuste. Se debe mostrar la matriz de confusión con distintos colores según el valor del *accuracy*.

3) Usando la mejor red encontrada en validación (aquella con mayor *accuracy* en validación), calcular la matriz de confusión normalizada y el *accuracy* normalizado, **usando el conjunto de prueba**.

4) Análisis de los resultados obtenidos:
   a) Explicar los efectos de variar la cantidad de neuronas en la capa oculta y cómo esto afecta el desempeño de la red.
   b) Explicar los efectos de variar la cantidad de capas ocultas y cómo esto afecta el desempeño de la red.
   c) Explicar el efecto de la función de activación y cómo esto afecta el desempeño de la red.
   d) Analizar tiempos de entrenamiento, matrices de confusión y *accuracies* de las arquitecturas probadas en validación.
   e) Analizar la matriz de confusión y el *accuracy* en el conjunto de prueba, respecto a los obtenidos en el conjunto de validación.

Los informes y códigos deben ser subidos a u-cursos a más tardar a las 23:59 del día Lunes 3 de Junio. Incluir un corto archivo de texto explicando cómo se utiliza su programa. Las tareas atrasadas serán penalizadas con un punto base más un punto de descuento adicional por cada día de atraso, incluyendo los fines de semana.

Importante:

- La evaluación de la tarea considerará el correcto funcionamiento del programa, la inclusión de los resultados de los pasos pedidos en el informe, la calidad de los experimentos realizados y de su análisis, la inclusión de las partes importantes del código en el informe, así como la forma, prolijidad y calidad del mismo. Se debe seguir la estructura indicada en material docente.
- Las partes importantes del código (sobre todo las modificadas/programadas) así como los resultados deben ser incluidas en el informe.
- Incluya el código completo como un anexo al final del informe.
- Cada uno de los puntos pedidos debe estar documentado en el informe
- El informe se debe entregar en formato PDF, y los códigos dentro del informe se deben agregar como texto (no como imágenes obtenidas usando un pantallazo). El formato básico de los informes se indica en el material docente de u-cursos.
- El código se debe entregar como un notebook con extensión .ipynb
- Se debe activar el uso de GPU en colaboratory para acelerar el proceso de entrenamiento.
- El informe se debe subir a la plataforma turnitin, en formato PDF. Tanto el contenido como los códigos del informe deben estar en formato de texto (no pantallazos).

El link de turnitin para la tarea 4 es:

https://uchile.turnitin.com/originality/hand-in-link?jwt=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJyZWR3b29kSWQiOiIzYzgxZGQ2Yi01OWE2LTQ2MGEtODE2OC01MGI0OWM1Y2YyYTEiLCJleHAiOjE3MjA2MjE4MDAsImlhdCI6MTcxNTk1NjE2NSwianRpIjoiZTkxNWVmMTgtMDQzMS00YWVkLTllNGQtNzgyZDdiYTUyNmUyIiwidGVuYW50IjoidWNoaWxlIiwiZm9sZGVySWQiOiIxNWNjOTRhZC0wYzdkLTQ5N2MtYWJmYS1kMmE1NzBmYWM0MzUifQ.J4HO1xBcgfItz9rmhDfilWfEXKVNnnhs0kDY6juRFxk